

WasteWise: A Multi-Channel IoT Architecture for Real-Time Food Waste Monitoring and Analytics Across Institutional Hostel Messes

Gaurav Deori, Nishanka Borthakur, Latifun Nisha, Parismita Sonowal, Ashish Kumar Mahato

Technology Innovation Hub (TIH), Indian Institute of Technology Guwahati

TIDF Research Building, IIT Guwahati, Assam 781039, India

deorigaurav1400@gmail.com

Abstract—Food waste in institutional kitchens is large, recurrent and almost entirely unmeasured, and what is not measured cannot be managed. Hostel messes are nonetheless well suited to instrumentation: waste arrives at a small number of known disposal points, at predictable times, in an indoor environment with mains power and network coverage. This paper presents WasteWise, a deployed four-layer IoT platform that measures, aggregates and analyses food waste across the fourteen residential hostels of IIT Guwahati. Each sensing node pairs a strain-gauge load cell with an HX711 24-bit bridge amplifier and an ESP32 microcontroller; an event-driven firmware state machine emits exactly one record per disposal event, carrying weight, hostel identifier and link-quality telemetry. Readings are published directly to a sharded set of four ThingSpeak channels, an arrangement that quadruples aggregate write capacity under a free-tier rate limit of one update per channel per fifteen seconds and, critically, keeps data capture alive when the application server is not. A Node.js/Express gateway holds every cloud credential server-side, fans out reads across the four channels concurrently, and enforces privileged operations through middleware rather than through hidden interface elements. A single-page dashboard converts raw mass into the units in which decisions are made — kilograms, rupees, kilograms of CO₂e and grams per resident — and supports both an administrative and a read-only role. We further describe an advertising-funded hostel kiosk layer intended to make the deployment financially self-sustaining. We report end-to-end functional verification of nine security- and data-path behaviours, an analytical characterisation of the sensing chain giving an estimated noise-limited resolution of approximately 3.4 g, and an ingestion-capacity analysis showing that channel sharding raises the ceiling from 240 to 960 events per hour. Field rollout of the fourteen nodes is in progress and is reported honestly as such.

Keywords—*Internet of Things; food waste management; ESP32; HX711 load cell; ThingSpeak; real-time dashboard; smart campus; sustainability analytics*

I. INTRODUCTION

Food waste is simultaneously an economic loss, a social failure and an environmental burden, and it is unusual among sustainability problems in that the intervention is neither technically difficult nor expensive — it is simply invisible. The United Nations Environment Programme estimates that Indian households alone discard on the order of 68.7 million tonnes of food each year, roughly 50 kg per person, placing India second only to China in absolute household food waste [1]. Estimates of the associated national economic loss run to approximately ₹89,000 crore annually, close to one per cent of gross domestic product [2]. Beyond the direct cost, discarded food carries the

embedded water, energy, fertiliser and transport of everything that was required to produce it, and decomposes anaerobically in landfill to emit methane.

Within this broad problem, institutional kitchens — hostel messes, college canteens, corporate cafeterias, hotel banquets — are the highest-leverage place to intervene, for two reasons. First, they are centralised: thousands of meals a day follow from a single planning decision, so a small correction to preparation quantity compounds across the entire population served. Second, and more important for an engineering intervention, they are measurable. Waste arrives at a small number of known disposal points, at predictable times, in a controlled indoor environment with mains power and Wi-Fi coverage. That is precisely the condition under which a fixed sensing network is cheap and reliable, and it is why a campus is a natural proving ground for a system intended to generalise to hotels and restaurants.

The core gap is not a lack of willingness but a lack of evidence. Where records exist at all they are manual registers: written inconsistently, aggregated rarely, and almost never fed back into the next week's menu. A mess manager asked how much was wasted at lunch yesterday, which hostel is worst, or whether last month's menu change helped, generally cannot answer. Absent that evidence, no corrective action can be planned, and no action that is taken can be evaluated.

A substantial body of IoT work has developed the sensing techniques relevant to this problem, and we survey it in Section II: load-cell weighing at the bin [3], [4], ultrasonic fill-level ranging [5], [6], lightweight publish/subscribe alerting [8], and sensor-driven spoilage classification [9]. Building on that foundation, the system described here is directed at a specific institutional requirement — a continuous, campus-wide record of how much food is discarded, when, by which kitchen, and what it is worth — and at the systems-level questions that a multi-site deployment raises: ingestion capacity when many nodes report simultaneously, credential handling once a dashboard is exposed to real users, and the recurring cost of keeping the deployment running.

This paper describes WasteWise, a system built and deployed at IIT Guwahati under its Technology Innovation Hub. The specific contributions are as follows.

- *A campus-scale formulation.* Fourteen hostel messes are instrumented as one measurement network with a shared identifier scheme, so that comparison between kitchens — the analysis that actually changes catering behaviour — is a first-class operation rather than an afterthought.
- *An event-driven sensing firmware.* A five-state machine on the ESP32 converts a continuous load signal into exactly one clean record per act of disposal, eliminating the duplicate-suppression and noise-filtering burden that a streaming design would push into the cloud.
- *Multi-channel cloud sharding.* Hostels are partitioned across four ThingSpeak channels with a symmetric mapping implemented at both the firmware and the server. We give the capacity analysis that motivates this and show it raises the aggregate ingestion ceiling fourfold under an unchanged free-tier rate limit.
- *Server-side credential isolation and enforcement.* No cloud credential of any kind reaches the browser, and privileged operations are gated by middleware that we verify by direct adversarial request, not merely by hiding interface controls.
- *Decision-grade analytics.* Raw mass is converted into monetary, carbon and per-capita units, with every conversion factor declared, tunable, and explicitly labelled as an estimate rather than a measurement.
- *An advertising-funded sustainability model.* Hostel-mess display kiosks alternate live waste feedback with scheduled paid advertisements, generating revenue intended to cover the recurring cost of the deployment.

The remainder of the paper is organised as follows. Section II surveys the background literature. Section III presents the layered architecture. Sections IV and V describe the sensing hardware and firmware, Section VI the cloud ingestion layer and its capacity analysis, Section VII the backend gateway and Section VIII the analytics dashboard. Section IX describes the self-sustaining deployment model, Section X the security design and its honest open issues, and Section XI the evaluation. Sections XII to XIV give limitations, future work and conclusions.

II. BACKGROUND AND RELATED WORK

This section surveys the sensing and communication techniques on which the present system builds. Four strands of work are relevant: weighing at the point of disposal, fill-level ranging, prevention-side estimation with lightweight messaging, and sensor-driven spoilage classification.

A. Weighing at the point of disposal

The load-cell-plus-microcontroller sensing chain adopted in this work is well established in the literature. Gayathri et al. [3] describe a system in which a servo-actuated lid opens on presentation of a management-issued RFID card; an Arduino reads a load cell, an LCD reports the recorded mass immediately, and records accumulate in a MySQL database behind a web interface with separate user and administrator logins. Jeyanthi et al. [4] employ a comparable arrangement

built around an HX711 amplifier, adding an ultrasonic level sensor for bin fullness and a GSM modem that texts the mess supervisor when the bin requires emptying. A particularly useful observation reported in [4] is that repeated high waste on the same weekday indicates a fault in the menu rather than in individual behaviour; this directly motivates the day-of-week and meal-wise analytics described in Section VIII.

B. Fill-level monitoring and collection logistics

A second strand treats the container as the object of measurement. Kumar et al. [5] report an ultrasonic smart bin that alerts a municipal server when fill level crosses a threshold; Misra et al. [6] generalise this to a cloud-monitored waste management system, and Haque et al. [7] extend the idea to route efficiency for collection vehicles. This work optimises the logistics of removing waste, and it establishes the practical value of a cloud-held time series that operational staff can query at any time. Because the density of wet food waste varies widely with composition, volume is an imprecise proxy for mass. The present system therefore measures mass directly at the point of disposal, mass being the quantity that can be priced, carbon-costed and fed back into preparation planning, and treats fill level as an operational convenience.

C. Prevention-side estimation and messaging

Sharmitha and Karthikeyan [8] address the problem upstream of disposal, estimating the quantity of food required for an expected headcount, monitoring buffet container levels with ultrasonic sensors to trigger just-in-time refilling, and prompting donation of surplus through a synced list of NGO contacts once an event overruns its scheduled close. Alerting is carried over MQTT, whose publish/subscribe model suits low-latency notification and is characterised against other machine-to-machine protocols in [14] and [15]. Estimation of this kind depends on a longitudinal record of what was actually discarded in order to be calibrated over time, and producing exactly such a record is the purpose of the system described here.

D. Quality and spoilage monitoring

A fourth strand addresses waste arising from spoilage rather than over-preparation. Vasanthi et al. [9] instrument stored food with temperature, humidity, gas and pH sensors on an ESP8266 and classify the resulting feature vector using logistic regression, flagging deterioration before it becomes waste. Related work applies computer vision to estimate food volume and leftover quantity [16]. This strand is methodologically instructive for the present work in one respect: it demonstrates that a supervised model over a modest multi-sensor feature vector is tractable on this class of hardware, which supports the roadmap item of forecasting preparation quantity from historical disposal records set out in Section XIII.

E. Techniques carried forward

From this body of work the present system carries forward the load cell and HX711 sensing chain characterised in [3] and [4], the cloud-held queryable time series of [5] and [6], and the menu-feedback interpretation of recurring weekday peaks

noted in [4]. The design elements specific to a fourteen-site institutional deployment — event-driven measurement at the node, sharded cloud ingestion, server-side credential isolation

and a self-sustaining funding mechanism — are developed in the sections that follow.

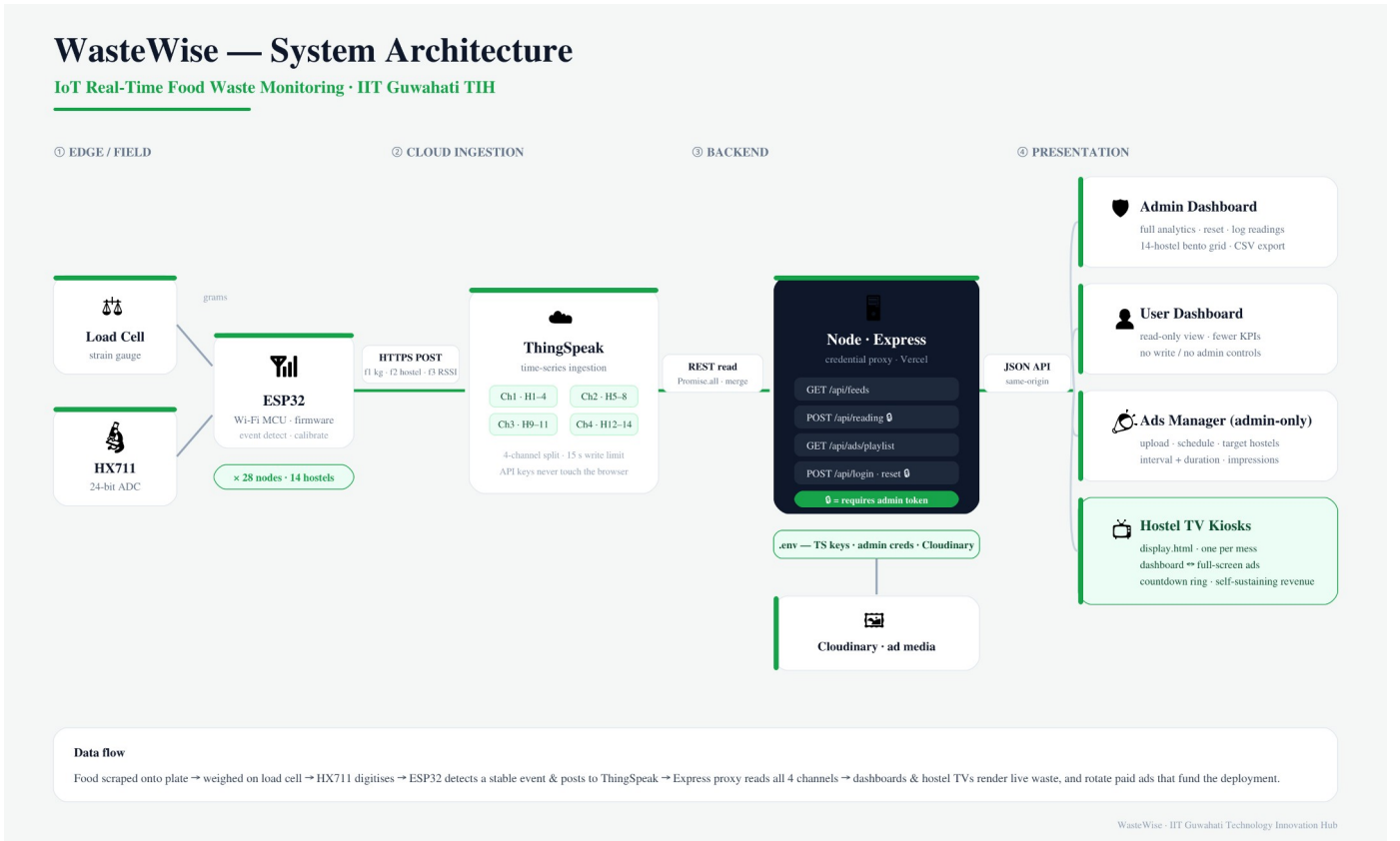


Fig. 1. WasteWise four-layer architecture. Sensing nodes publish directly to sharded cloud channels over HTTPS, so measurement continues independently of the application server. The Node/Express gateway is the sole holder of cloud credentials; the browser and the hostel kiosks receive only same-origin JSON. Endpoints marked with a lock require a valid administrator token.

III. SYSTEM ARCHITECTURE

A. Design principles

Four constraints shaped the architecture, and it is worth stating them explicitly because each one is visible in the resulting design.

- P1 — Device autonomy.** Measurement must not depend on the availability of the application server. A mess does not stop generating waste because a web service is being redeployed, and a system that loses those readings loses precisely the continuity that makes trend analysis possible.
- P2 — Zero client credentials.** No cloud API key may ever be delivered to a browser. This is stated as an absolute because the intermediate positions — obfuscation, short-lived embedding, read-only keys — all reduce in practice to publishing a working credential.
- P3 — Replaceable layers.** Each layer communicates with its neighbours through a narrow, documented interface, so that the cloud service, the gateway or the interface can be substituted without disturbing the others.
- P4 — Replicable node cost.** The sensing node must be inexpensive enough to build fourteen times immediately

and hundreds of times later, which rules out industrial weighing terminals and single-board computers at the edge.

B. Layer decomposition

Fig. 1 shows the resulting four-layer decomposition and Table I summarises the responsibilities. The consequence of P1 is the most architecturally significant: each ESP32 publishes to the cloud service directly rather than to our own server. The gateway is therefore a reader and analyser of data, never a collector of it. If the gateway is down, or the deployment is migrated between hosting providers, or the dashboard is rewritten, readings continue to be captured and timestamped without interruption and appear retrospectively when the gateway returns. This costs us the ability to apply server-side validation at ingest — a trade we accept, and compensate for with bounds checking at both the firmware and the presentation layer.

TABLE I. LAYER RESPONSIBILITIES

Layer	Technology	Responsibility
Sensing	Load cell, HX711, ESP32 (Arduino C+)	Detect a stable disposal event, convert raw counts to kilograms, publish over Wi-Fi with hostel identity and link telemetry

Layer	Technology	Responsibility
Cloud ingestion	ThingSpeak, 4 channels	Timestamp, persist and serve time-series readings; decouple device lifetime from server lifetime
Application	Node.js 18+, Express 4, dotenv	Hold all credentials, aggregate channels concurrently, authenticate administrators, expose a narrow JSON API, serve the static site
Presentation	HTML/CSS/vanilla JS, inline SVG	Compute analytics client-side, render the dashboard and kiosk views, enforce role-scoped visibility

C. End-to-end data path

A single measurement traverses the system as follows. Food is scraped onto the instrumented platform; the load cell converts the applied force to a differential voltage of a few millivolts; the

IV. SENSING NODE DESIGN

A. Component selection

The node is built from four commodity elements, listed with rationale in Table II. The ESP32 integrates a 2.4 GHz radio and a TLS stack on the same die as the application processor. This removes an inter-processor serial link and the AT-command firmware such a link requires, and — relevant to P2 — makes it practical to speak HTTPS rather than plaintext HTTP from the edge. An on-die Wi-Fi radio also avoids a recurring per-node SIM subscription, which matters at fourteen nodes and matters considerably more at a hundred.

TABLE II. SENSING NODE BILL OF MATERIALS AND RATIONALE

Component	Function	Selection rationale
ESP32 Dev Module	MCU and Wi-Fi radio	Integrated 2.4 GHz radio and TLS stack; no external network coprocessor; no recurring SIM cost
HX711 amplifier	24-bit bridge ADC	Purpose-built for resistive bridge sensors; resolves the microvolt-scale load-cell signal; simple two-wire interface
Strain-gauge load cell	Force to voltage	Low cost, mechanically simple, adequate accuracy at kitchen scale
Bin platform	Receives disposal	Fits the existing mess workflow; requires no behaviour change from residents

B. Signal chain and resolution

A load cell cannot be read directly by a microcontroller: its output is a differential signal of a few millivolts at full scale, well below the resolution of the ESP32's own ADC and far below its noise floor. The HX711 resolves this with a low-noise programmable-gain amplifier followed by a 24-bit sigma-delta converter, which is why this pairing is the standard construction for a digital scale.

It is useful to establish analytically what resolution this chain can support, since that determines whether the firmware thresholds in Section V are physically meaningful. For a load cell of rated capacity C with sensitivity S (in mV/V) excited at voltage V , the full-scale differential output is

$$V_{es} = S \times V \quad (1)$$

and the voltage produced per unit mass is V_{es} / C . Taking a representative 50 kg cell with $S = 1$ mV/V excited at $V = 3.3$ V gives $V_{es} = 3.3$ mV, hence 66 μ V per kilogram. The HX711 at gain 128 exhibits an input-referred noise of approximately 0.5

HX711 amplifies and digitises it to 24 bits; the ESP32 firmware waits for the reading to settle, converts counts to kilograms using a per-unit calibration constant, and issues an HTTPS GET carrying weight, hostel code and Wi-Fi RSSI to the channel that owns that hostel. The cloud service timestamps and stores the record. On the next dashboard poll — at most twenty seconds later — the gateway fetches all four channels concurrently, merges and time-sorts them, and returns a single uniform feed. The browser buckets readings by hostel and by meal window, applies the declared conversion factors, and re-renders. End-to-end latency from disposal to display is bounded by the firmware settling window, the cloud write interval and the dashboard poll period, and is dominated by the last of these.

μ V RMS at its 10 samples/s output rate. The noise-limited resolution of a single conversion is therefore

$$\Delta m = \sigma_n \times C / (S \times V) \approx 7.6 \text{ g} \quad (2)$$

and the firmware's five-sample average improves this by $\sqrt{5}$ to approximately 3.4 g, assuming uncorrelated noise. This is comfortably finer than the 50 g event threshold and the 20 g stability band used in Section V, which confirms that those parameters are set by mechanical and behavioural considerations — vibration, a plate being set down and lifted — rather than by converter noise. It also indicates that pursuing finer thresholds would not be productive: below roughly 20 g the limiting factor is the mechanics of the platform, not the electronics.

C. Interconnection

Table III gives the interface. The load cell connects to the HX711 bridge inputs using the conventional colour code: E+ and E− (excitation) to red and black, A+ and A− (signal) to white and green.

TABLE III. HX711 TO ESP32 INTERFACE

HX711 pin	ESP32 connection	Purpose
VCC	3V3	Supply
GND	GND	Common ground
DT (DOUT)	GPIO 16	Serial data from the converter
SCK (PD_SCK)	GPIO 4	Conversion clock driven by the MCU

D. Calibration

A load cell reports raw converter counts, not mass, and the constant relating the two depends on the individual cell, its mechanical mounting and the excitation voltage. Establishing that constant is the single most error-prone step in bringing up a scale, and in our experience it is also the step most often confused with a wiring fault. The firmware therefore ships a dedicated calibration mode, and separately performs a boot-time self-check.

- Set `CALIBRATION_MODE` = true, upload, and open the serial console at 115200 baud.
- Leave the platform empty; the firmware tares automatically over twenty averaged conversions.
- Place a reference mass of accurately known value, for example 500 g.

- Read the printed raw count R and compute the constant as $k = R / m$, in counts per gram.
- Enter k as `CALIBRATION_FACTOR`, set `CALIBRATION_MODE = false`, and re-upload.

As a worked example, a raw reading of 210,000 counts under a 500 g reference yields $k = 420.0$ counts per gram. The boot self-check is a separate mechanism: if the HX711 does not assert data-ready within 1.5 s, the console prints an explicit diagnostic naming the four pins to inspect. This distinguishes a wiring fault from a calibration fault, which is the distinction that consumed the most time during initial bring-up and which a bare numeric readout does not make.

V. FIRMWARE DESIGN

A. Event-driven measurement

The scale does not stream. A continuous feed would exceed the cloud write limit within seconds, and would in any case produce a signal requiring extensive post-hoc segmentation to recover the quantity of interest, namely how much food was thrown away in one act. Instead the firmware recognises a discrete disposal event through the five-state machine of Table IV.

TABLE IV. FIRMWARE EVENT-DETECTION STATE MACHINE

State	Condition and action
Idle	Platform empty; the load cell is polled at approximately 5 Hz
Detection	Mass exceeds <code>MIN_EVENT_KG = 0.05 kg</code> , rejecting vibration and airflow
Stabilisation	Reading must remain within <code>STABLE_TOL_KG = 0.02 kg</code> for <code>STABLE_SAMPLES = 8</code> consecutive conversions; a 6 s timeout prevents an unbounded wait
Transmission	The settled weight is posted with hostel code and RSSI
Lockout	No further transmission until the platform is cleared below the event threshold, guaranteeing exactly one record per disposal

The design yields records that are directly human-meaningful: one row per act of throwing food away, with a defensible weight, rather than a noisy waveform that must be de-duplicated and segmented downstream. It also moves the de-duplication logic to the only place that has the necessary information — the node knows whether the platform has been cleared, and the cloud does not.

B. Transmitted payload

Each transmission carries three fields, summarised in Table V. Including the received signal strength turns every data point into a health check as well as a measurement. A node whose link is degrading can be identified and relocated before it begins silently dropping readings, which is a practical necessity when fourteen unattended nodes are distributed across a large campus and no one is watching any individual one.

TABLE V. TRANSMITTED PAYLOAD

Field	Content	Units	Purpose
field1	Settled weight	kg	Primary measurement
field2	Hostel code (1–14)	integer	Attribution; drives all per-hostel analytics
field3	Wi-Fi RSSI	dBm	Device health surfaced in the Device Monitor

C. Rate-limit compliance

The cloud service's free tier accepts at most one update per channel per fifteen seconds, and rejects violations silently with a "0" response rather than an error. Silent rejection is the dangerous failure mode here, because it discards exactly the readings generated during a busy meal service — the ones that matter most. The firmware therefore enforces a 16 s minimum inter-transmission interval locally and, if an event completes sooner, blocks and reports the wait on the console rather than issuing a request that will be dropped. A "0" response, should one still occur, is reported together with the two likely causes.

D. Transmission procedure

Algorithm 1 summarises the transmit path, including the rate-limit interlock and the channel-selection function discussed in Section VI.

Algorithm 1: Transmit a settled reading

```

1: procedure SEND(kg, hostel)
2:   if Wi-Fi not connected then CONNECT()
3:   if Wi-Fi still not connected then return false
4:    $\delta \leftarrow \text{now} - \text{lastSend}$ 
5:   if  $\text{lastSend} \neq 0$  and  $\delta < 16000$  ms then
6:     WAIT(16000 -  $\delta$ )  $\triangleright$  avoid drop
7:   key  $\leftarrow$  WRITEKEY(hostel)  $\triangleright$  shard select
8:   if key =  $\emptyset$  then return false  $\triangleright$  bad code
9:   url  $\leftarrow$  endpt + key + f1=kg + f2=hostel + f3=RSSI
10:  code, body  $\leftarrow$  HTTPS_GET(url)
11:  lastSend  $\leftarrow$  now
12:  return (code = 200) and (body  $\neq$  "0")
13: end procedure

```

VI. CLOUD INGESTION AND CHANNEL SHARDING

ThingSpeak was selected as the ingestion service. It provides timestamped time-series storage, a REST interface and a free tier adequate for a pilot, with no server administration and no database to operate — which, for a student-built system that must survive team turnover, is a more important property than raw capability.

A. Capacity analysis

A single channel would have been a hard bottleneck, and quantifying it is what motivated the sharded design. Let r be the permitted write rate per channel, n the number of channels, and H the number of hostels. With the free-tier limit of one update per fifteen seconds,

$$r = 1 / 15 \text{ s} = 240 \text{ writes} \cdot \text{h}^{-1} \quad (3)$$

so the aggregate ingestion ceiling is

$$R = n \times r = 960 \text{ writes} \cdot \text{h}^{-1} \text{ for } n = 4 \quad (4)$$

and the ceiling available to any one hostel, assuming fair contention within its shard, is

$$r_h = r \times n / H \approx 68.6 \text{ events} \cdot \text{h}^{-1} \quad (5)$$

against 17.1 events per hour for a single shared channel. A second, independent bound comes from the node itself: the firmware's 16 s interlock caps one node at 225 events per hour regardless of channel capacity. Sharding therefore moves the binding constraint away from the shared channel and towards the node, which is the correct place for it — a node-local limit degrades gracefully and predictably, whereas channel contention produces silent, correlated loss across several hostels at once, concentrated in the busiest minutes.

We note honestly that neither bound is generous relative to the theoretical worst case. A 450-resident hostel in which every resident disposed separately within a one-hour meal window would require 450 events per hour, well above r_h . In practice disposals cluster and many residents leave nothing, and the pilot has not yet run long enough at full deployment to establish the true event rate. Section XII records this as an open limitation, and the mitigation — a paid tier or self-hosted ingestion — is a configuration change rather than a redesign, precisely because of principle P3.

B. Hostel-to-channel mapping

Table VI gives the partition. The mapping is implemented symmetrically at both ends: in the firmware, a lookup function returns the correct write key for the board’s hostel code, so a node targets its own shard automatically and a technician flashing a board sets one constant; on the gateway, the inverse function routes manual writes, while reads fan out across all four shards. Adding a hostel therefore requires editing exactly two mappings and no protocol.

TABLE VI. HOSTEL-TO-CHANNEL PARTITION

Shard	Codes	Hostels
Channel 1	1–4	Brahmaputra, Subansiri, Lohit, Dhansiri (Girls)
Channel 2	5–8	Manas, Kopili, Barak, Jia Bharali
Channel 3	9–11	Pagladiya, Dikhow, Dhansiri (Boys)
Channel 4	12–14	Kojoli Annex, Bhrigu, Neelachal

VII. BACKEND GATEWAY

The gateway is an Express application of approximately 300 lines with three responsibilities: hold secrets, aggregate the shards, and gate privileged actions. It also serves the static front end, so one process runs the entire application and one deployment step suffices.

A. Interface

Table VII gives the API surface. It is deliberately narrow. The browser cannot express a request to the cloud service that the gateway has not explicitly implemented, which converts credential protection from a matter of client discipline into a structural property.

TABLE VII. GATEWAY API SURFACE

Endpoint	Auth	Description
GET /api/config	public	Non-secret state: channel identity, whether writes and admin auth are configured
GET /api/feeds	public	Merged, time-sorted readings from all four shards with the reset filter applied
POST /api/login	public	Validates administrator credentials server-side; returns a token, or HTTP 401
POST /api/reading	admin token	Writes a reading, routed to the shard owning the given hostel
POST /api/reset	admin token	Clears dashboard history; optionally wipes cloud channels
GET /api/ads/playlist	public	Ordered advertisement playlist for a given hostel kiosk
POST /api/ads	admin token	Uploads and schedules a campaign

B. Concurrent shard aggregation

Because the fourteen hostels are spread over four channels, the feeds endpoint issues four HTTPS requests concurrently, concatenates the responses, sorts the union chronologically, applies the reset filter and truncates to the requested number of most-recent records. Fetching in parallel rather than sequentially bounds the latency by the slowest shard rather than by their sum,

$$T_{par} = \max_i t_i + t_{merge} \text{ vs. } T_{seq} = \sum_i t_i \quad (6)$$

so that dashboard latency remains close to that of a single channel as the shard count grows. Each request carries a ten-second timeout, so one unresponsive shard degrades the response rather than hanging it. The client receives one uniform feed and is entirely unaware of the sharding — which is what allows the shard count to change later without touching the front end.

C. Two-tier reset semantics

Clearing stored feeds from a channel requires the account-level user key, which is substantially more privileged than the per-channel read and write keys. Rather than requiring that key unconditionally, the reset endpoint implements two tiers. A soft reset, always available, records the reset instant to a marker file and thereafter filters out every reading older than that moment; the dashboard starts clean immediately, the marker survives a restart, and the historical data remains in the cloud, so the action is recoverable by deleting the marker. A hard reset, available only when the account key is configured, additionally issues a delete to each shard and is irreversible. The endpoint reports which tier actually executed, so an operator is never left believing the cloud was wiped when only the local view was cleared. The everyday need — a clean demonstration — is satisfied without risk, and the destructive capability is opt-in.

VIII. ANALYTICS AND PRESENTATION

The dashboard is a single-page application in plain HTML, CSS and JavaScript with no framework and no build step; every chart is hand-generated inline SVG. This keeps the deployable artefact to a handful of static files, removes an entire class of toolchain maintenance, and leaves the project readable by future contributors who have no framework expertise — a real consideration for a system that must be handed between successive student cohorts. It polls the gateway every twenty seconds.

A. Layout and scoping

The interface uses a bento grid: a status bar, vertical rails listing all fourteen hostels, and a central analytics column, as shown in Fig. 2. Every hostel card carries today’s mass, an efficiency score, a seven-day sparkline and a live/offline indicator, and is selectable — choosing one re-scopes the entire centre column to that hostel, and an overview control restores the campus view. This single scoping control is what makes cross-hostel comparison a routine operation rather than a data-export exercise.

B. Derived metrics

Raw mass is not the unit in which catering decisions are made. The dashboard therefore converts it, using the factors of Table VIII, all declared in a single configuration file and tunable without code changes.

TABLE VIII. DERIVED METRICS AND CONVERSION FACTORS

Metric	Formula	Constant
Value lost (₹)	$m \times \text{PRICE_PER_KG}$	₹75 per kg
Carbon (kg CO ₂ e)	$m \times \text{CO2_PER_KG}$	2.5 per kg
Per-capita waste (g)	$m \times 1000 / \text{residents}$	450 per hostel
Meal equivalents	$m / \text{KG_PER_MEAL}$	0.4 kg per meal
Efficiency score (%)	$\text{clamp}(100 - g / 1.6, 40, 99), g \text{ in } g/\text{resident}$	derived
High-waste flag	$\text{today} > 1.3 \times 7\text{-day mean}$	derived

We are explicit about the epistemic status of these figures, because conflating them with measurements is the most likely way for this system to mislead. Only mass, timestamp, hostel identity and link quality are measured. The monetary and carbon values are planning estimates produced by a single campus-wide multiplier; their purpose is to express waste in

units a warden or caterer can act on, not to serve as an audited account. The efficiency score is a normalised internal indicator for ranking hostels against one another, not a physical quantity, and comparisons of it across institutions with different populations would be meaningless. The interface labels estimated quantities as estimated.

C. Data integrity

A miscalibrated or faulty load cell can emit implausible values, and a single wild reading will visibly distort every aggregate on the page. The client therefore discards any reading outside 0.02 kg to 60 kg before it enters the statistics. This is a safety net against sensor faults, not a substitute for calibration, and we treat a reading rejected by these bounds as a signal to inspect the node rather than as noise to be forgotten. A demonstration switch allows hostels awaiting hardware to be populated with deterministic simulated data so that the complete interface can be shown before rollout finishes; simulated hostels are visually marked, and the switch is disabled for any presentation of real results.

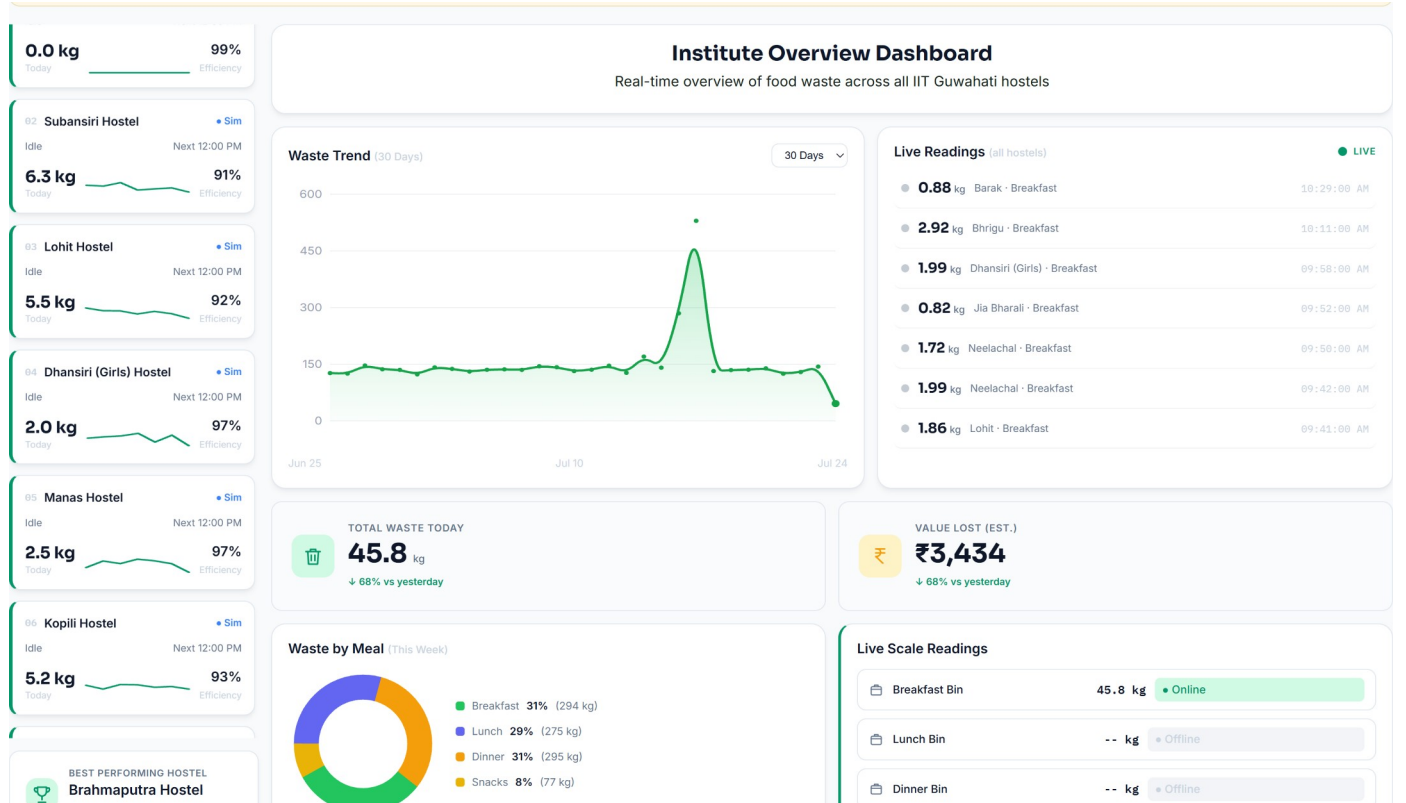


Fig. 2. Campus overview view of the analytics dashboard. Left: per-hostel cards with today’s mass, efficiency score and seven-day sparkline. Centre: thirty-day trend, aggregate mass and estimated value, and the weekly meal-wise split. Right: the live per-reading feed, which preserves individual disposal events and was instrumental in diagnosing a calibration fault that aggregate views concealed. Hostels awaiting hardware are marked "Sim".

IX. AN ADVERTISING-FUNDED DEPLOYMENT MODEL

Campus IoT pilots characteristically end not in technical failure but in funding exhaustion: the grant covers the hardware, and nothing covers the connectivity, replacement parts, hosting and student time in year two. We treat funding continuity as a

genuine engineering requirement rather than an administrative afterthought.

WasteWise addresses it with a display layer that is simultaneously a feedback mechanism and a revenue mechanism. A kiosk view, served from the same origin as the dashboard, runs on an existing television in each hostel mess. It

alternates between a live waste panel — today’s mass for that hostel, its standing among the fourteen, and the running trend — and full-screen scheduled advertisements from local businesses, with a countdown ring indicating when the waste panel returns. Campaigns are uploaded through an administrator-only endpoint, stored on an external media service, and targeted per hostel with a configurable rotation interval and duration; the kiosk requests an ordered playlist for its own hostel and reports playback so that impressions can be counted.

Two properties of this arrangement are worth noting. First, the feedback and the funding reinforce rather than compete with one another: the same screen placement that makes advertising valuable — a captive audience at mealtimes — is exactly the placement at which waste feedback is most likely to change behaviour, and residents see their own hostel’s figure at the moment they are deciding how much to take. Second, the revenue scales with the deployment: each additional instrumented mess adds both a measurement point and an advertising surface, so growth does not deepen the funding deficit. We report the mechanism as implemented and operational; we do not yet report revenue figures, as no commercial campaign has run for long enough to produce meaningful ones.

X. SECURITY MODEL

A. Credential isolation

The system originally embedded cloud keys directly in client-side JavaScript, which meant every visitor received working credentials. We record this because it is the default outcome whenever the device and the interface talk to the cloud service directly, and because the correction is what motivated the gateway layer. All keys now reside in an environment file excluded from version control and never served; the static handler explicitly denies dotfiles, so neither the environment file nor the repository metadata is reachable over HTTP. The browser holds no cloud credential and can reach the cloud service only through the gateway’s narrow API.

B. Roles and server-side enforcement

Two roles are supported. An administrator is verified server-side against credentials held in the environment and receives a token; the role has access to the full dashboard, device monitor, alerts, recommendations, CSV export, manual logging, campaign management and reset. A viewer is a read-only account with a simplified dashboard. Privileged endpoints are wrapped in middleware that demands a valid token header and returns HTTP 403 otherwise. The distinction we insist on is that hiding a control is a usability measure and the middleware is the actual access control; accordingly we verified the latter by issuing writes directly with no token and while authenticated as a viewer, rather than by observing that the button was absent.

C. Open issues

The following are genuine weaknesses in the current build, and we state them rather than omit them because a deployed system described only by its intended design is not usefully described at all.

- *Administrator credentials in client configuration.* To make login work on a hosted deployment without configuring server environment variables, an administrator entry including a password and token was reintroduced into the client configuration file, which ships to every browser. This must be removed and server-only verification restored before any public deployment.
- *Credential history.* Cloud keys were committed early in development. History was subsequently rewritten to purge them, but any key that was ever public must be treated as compromised and regenerated.
- *Static shared token.* The administrator token is a fixed secret rather than a per-session expiring credential; it cannot be revoked individually or timed out.
- *Cosmetic viewer authentication.* The viewer account is checked in the browser and protects nothing. Read endpoints are unauthenticated by design, so this is a presentation convenience, not a control.

The remediation is straightforward and is listed in Section XIII: remove client-side administrator credentials in favour of environment variables, regenerate every cloud key, and replace the static token with signed, expiring sessions.

XI. EVALUATION

We report three classes of result and are explicit about what each does and does not establish. Functional verification exercises the data path and the access controls against the running system. Analytical characterisation bounds the sensing chain and the ingestion capacity. Field results from a complete fourteen-node deployment are not yet available and are not claimed.

A. Functional verification

Table IX lists behaviours exercised directly against the running system rather than assumed. Each was performed by issuing the stated request and observing the response, with adversarial cases — writes without a token, writes as a viewer, requests for the environment file — included deliberately.

TABLE IX. END-TO-END FUNCTIONAL VERIFICATION

Behaviour	Method	Result
End-to-end data path	Device-style write, read back through the feeds endpoint	Weight, hostel and RSSI returned intact and rendered
Shard aggregation	Concurrent fetch of all four channels	Single unified, time-sorted feed returned
Write authorisation	POST reading with no token	HTTP 403 — rejected
Administrator login	POST login, wrong then correct password	HTTP 401, then token issued
Viewer cannot write	Forced write while authenticated as viewer	HTTP 403 — rejected server-side
Role-scoped interface	Sign in as viewer, then as administrator	Admin-only panels hidden, then restored
Reset authorisation	POST reset with no token	HTTP 403 — rejected

Behaviour	Method	Result
Secret exposure	Request the environment file over HTTP	Not served
Credential isolation	Inspect browser network traffic	No request to the cloud service; only same-origin calls

B. Sensing chain characterisation

Table X collects the analytical bounds derived in Section IV-B together with the firmware thresholds they justify. We stress that the resolution figures are derived from datasheet noise specifications and stated load-cell parameters, not from a measured repeatability study against certified masses; such a study is planned as part of node commissioning and is listed in Section XIII. The purpose of the derivation here is to establish that the chosen thresholds are physically sensible, which it does with roughly an order of magnitude of margin.

TABLE X. SENSING CHAIN BOUNDS AND FIRMWARE THRESHOLDS

Quantity	Value	Basis
Full-scale bridge output	3.3 mV	1 mV/V at 3.3 V excitation
Sensitivity	66 μ V per kg	50 kg rated capacity
Converter input noise	$\approx 0.5 \mu$ V RMS	HX711, gain 128, 10 SPS
Single-sample resolution	≈ 7.6 g	Derived, (2)
5-sample averaged resolution	≈ 3.4 g	Derived, $\sqrt{5}$ improvement
Event threshold	50 g	Firmware, set by mechanics
Stability band	20 g	Firmware, set by mechanics
Accepted reading range	0.02–60 kg	Client-side validity bounds

C. Ingestion capacity

The sharded arrangement raises the aggregate ingestion ceiling from 240 to 960 events per hour, and the per-hostel ceiling under fair contention from 17.1 to 68.6 events per hour, as derived in (3)–(5). The node-local interlock independently caps a single node at 225 events per hour. Sharding therefore relocates the binding constraint from a shared resource, where contention causes correlated silent loss across several hostels during peak service, to a node-local resource, where the failure is isolated, predictable and observable. This relocation, rather than the fourfold headroom itself, is the substantive result.

D. Deployment status and an instructive failure

The software platform is complete and operational: the shards are provisioned, the gateway aggregates them, the dashboard renders live readings with role separation, and the firmware including calibration and self-check tooling is complete. The remaining work is physical: calibrating and installing a node in each of the fourteen messes. We state plainly that the campus-wide waste figures a full deployment will produce are not yet available, and that Fig. 2 shows hostels awaiting hardware marked as simulated.

One observation from testing is worth reporting because it validates a design decision. A day of test traffic produced an aggregate far above the surrounding days. The daily total alone gave no indication of the cause and would plausibly have been read as a genuine spike in waste. The per-reading live feed, however, showed the aggregate was composed of individual readings whose magnitude was implausible for a single disposal, which immediately identified an uncalibrated load cell

rather than a behavioural change or a software fault. This is the case for retaining per-event visibility alongside aggregates, and for the validity bounds of Section VIII-C: an aggregate view alone would have concealed a sensor fault behind a plausible-looking number, which is a considerably worse failure than an obviously wrong one.

XII. LIMITATIONS

- *No category discrimination.* A load cell measures mass only and cannot distinguish rice from vegetables from bone. Menu-level rather than quantity-level optimisation would require an operator selection at the bin or an additional sensing modality.
- *Ingestion ceiling.* Sharding eases but does not remove the free-tier rate limit, and the per-hostel ceiling of 68.6 events per hour is below the theoretical worst case for a 450-resident hostel. Campus-wide rollout may require a paid tier or self-hosted ingestion.
- *Manual per-unit calibration.* Each scale is calibrated individually, and drift or mechanical disturbance requires recalibration. No automatic drift detection exists.
- *Network dependence.* A node without connectivity cannot deliver readings. The firmware retries but does not yet buffer events to local storage, so a sustained outage loses data.
- *Estimated economics.* Monetary and carbon values use fixed campus-wide multipliers and are indicative planning figures, not audited accounts.
- *Demonstration-grade authentication.* As detailed in Section X-C.
- *No measured accuracy study.* Resolution is derived analytically; a repeatability and linearity study against certified masses has not yet been performed.

XIII. FUTURE WORK

The immediate priorities are remediation and rollout: removing administrator credentials from client configuration in favour of server environment variables, regenerating every cloud key, and calibrating and installing the fourteen nodes with a documented repeatability check against certified masses at commissioning. In the short term we intend to add offline buffering to the firmware so that a network outage delays rather than destroys readings, a physical tare control and enclosure for field robustness and easy re-zeroing between meals, and threshold alerting by email or SMS so that action reaches a warden without anyone watching a dashboard. Medium-term work includes signed, expiring sessions with per-user accounts to replace the static token, and capture of a coarse waste category at the bin, which would move the analytics from how much to what and thereby address the most consequential limitation above. Longer term, the accumulated per-meal record is exactly the training set required for predictive preparation guidance — forecasting demand so that waste is prevented at source rather than measured after the fact — and the methodological precedent of [8] and [9] suggests this is

tractable. Extension beyond the campus to hotels and restaurants is the eventual goal, and the advertising-funded model of Section IX is what we believe makes it viable without recurring grant support.

XIV. CONCLUSION

WasteWise demonstrates that the invisibility of institutional food waste is an addressable engineering problem, and that addressing it at institutional scale requires attention to systems-level concerns beyond the sensing node itself. Using inexpensive standard components — a strain-gauge load cell, an HX711 amplifier and an ESP32 — the system captures a clean, attributable record of every disposal and converts it, within seconds, into figures a decision-maker can act on: kilograms today, rupees lost, carbon emitted, which hostel is worst and when the peak occurs.

The contributions we would emphasise are architectural rather than componentwise. Sensing nodes publish directly to sharded cloud channels, so measurement survives application downtime and ingestion contention is relocated from a shared resource to a node-local one, quantified in (3)–(5). Cloud credentials were moved out of the browser entirely and privileged operations are enforced by middleware verified through direct adversarial request. Implausible sensor values are bounded, per-event visibility is retained alongside aggregates — a decision vindicated when it exposed a calibration fault that the aggregate view concealed — estimates are labelled as estimates, and the destructive reset defaults to a recoverable tier. An advertising-funded kiosk layer makes the feedback surface pay for the deployment that produces it.

Where weaknesses remain, most importantly the administrator credentials currently present in client configuration and the absence of a measured accuracy study, they are documented in Sections X-C, XI-B and XII rather than obscured. With the software platform complete and verified, the path towards the project's stated aim — a zero-food-waste institute, and a model replicable across campuses, hotels and restaurants — now runs through physical deployment: calibrating and installing the fourteen nodes, closing the security gaps, and beginning to feed the resulting evidence back into menu and production planning.

ACKNOWLEDGMENT

The authors gratefully acknowledge the Technology Innovation Hub, Indian Institute of Technology Guwahati, for supporting this work, and the hostel mess administrations of IIT Guwahati for their cooperation during system trials.

REFERENCES

- [1] United Nations Environment Programme, "Food Waste Index Report 2021," UNEP, Nairobi, 2021.
- [2] Times of Agriculture, "Food wastage in India: from farm to bin," 2021. [Online].
- [3] N. Gayathri, A. R. Divagaran, C. D. Akhilesh, V. M. Aswiin, and N. Charan, "IoT based smart waste management system," in Proc. 7th Int. Conf. Advanced Computing and Communication Systems (ICACCS), Coimbatore, India, 2021, pp. 2042–2046, doi: 10.1109/ICACCS51430.2021.9441819.
- [4] S. Jeyanthi, A. Arun Kumar, M. M. Muthu Rani, M. I. Rohini, and V. Divya Jeyashree, "IoT based food waste monitoring and controlling system," *J. Pharmaceutical Negative Results*, vol. 14, no. 3, pp. 586–591, 2023, doi: 10.47750/pnr.2023.14.03.073.
- [5] N. S. Kumar, B. Vijayalakshmi, R. J. Prarthana, and A. Shankar, "IoT based smart garbage alert system using Arduino UNO," in Proc. IEEE Region 10 Conf. (TENCON), Singapore, 2016, pp. 1028–1034.
- [6] D. Misra, G. Das, T. Chakraborty, and D. Das, "An IoT-based waste management system monitored by cloud," *J. Material Cycles and Waste Management*, vol. 20, no. 3, pp. 1574–1582, 2018.
- [7] K. F. Haque, R. Zabin, K. Yelamarthi, P. Yanambaka, and A. Abdelgawad, "An IoT based efficient waste collection system with smart bins," in Proc. IEEE 6th World Forum on Internet of Things (WF-IoT), 2020.
- [8] S. Sharmitha and P. Karthikeyan, "Smart food wastage management system in IoT with effective alerts using MQTT protocol," *Indian J. Food Engineering*, vol. 1, no. 1, pp. 6–12, Dec. 2021.
- [9] R. Vasanthi, K. Ashika, and D. Archana, "IoT-based food quality monitoring system using machine learning," in Proc. 2nd Int. Conf. Networking and Communications (ICNWC), Chennai, India, 2024, doi: 10.1109/ICNWC60771.2024.10537341.
- [10] S. Hong, S. Park, B. Lee, J. Lee, D. Jeong, and S. Park, "IoT-based smart garbage system for efficient food waste management," *The Scientific World Journal*, vol. 2014, art. 646953, 2014.
- [11] T. Kadus, P. Nirmal, and K. Kulkarni, "Smart waste management system using IoT," *Int. J. Engineering Research and Technology*, vol. 9, no. 4, May 2020.
- [12] H. N. Saha et al., "Waste management using Internet of Things (IoT)," in Proc. 8th Annu. Industrial Automation and Electromechanical Engineering Conf. (IEMECON), Bangkok, 2017, pp. 359–363.
- [13] D. S. Abdul Minaam, M. Abd-Elfattah, and M. A. S. Ali, "Design of an Internet of Things (IoT) network system for kitchen food waste management," *Int. J. Computer Science and Network Security*, vol. 18, no. 5, May 2018.
- [14] P. Thota and Y. Kim, "Implementation and comparison of M2M protocols for Internet of Things," in Proc. Int. Conf. Applied Computing and Information Technology, 2016, pp. 43–48, doi: 10.1109/ACIT-CSII-BCD.2016.021.
- [15] R. A. Atmoko, R. Riantini, and M. K. Hasin, "IoT real time data acquisition using MQTT protocol," *J. Physics Conf. Series*, vol. 853, no. 1, art. 012003, May 2017.
- [16] J. Dehais, M. Anthimopoulos, S. Shevchik, and S. Mougiakakou, "Two-view 3D reconstruction for food volume estimation," *IEEE Trans. Multimedia*, vol. 19, no. 5, pp. 1090–1099, 2017.
- [17] AvanteK/Avia Semiconductor, "HX711 24-bit analog-to-digital converter for weigh scales," datasheet, rev. 2.0.
- [18] B. Necula, "HX711 Arduino Library." [Online]. Available: <https://github.com/bogde/HX711>
- [19] Espressif Systems, "ESP32 Arduino core documentation." [Online]. Available: <https://docs.espressif.com>
- [20] The MathWorks, Inc., "ThingSpeak REST API documentation — channel read, update and clear operations." [Online]. Available: <https://www.mathworks.com/help/thingspeak/>
- [21] Natural Resources Defense Council, "Wasted: how America is losing up to 40 percent of its food from farm to fork to landfill," NRDC Report, 2017.
- [22] Food and Agriculture Organization of the United Nations, "Food wastage footprint: impacts on natural resources," FAO, Rome, 2013.